SparseDet: A Simple and Effective Framework for Fully Sparse LiDAR-Based 3-D Object Detection

Lin Liu, Ziying Song, Qiming Xia, Feiyang Jia, Caiyan Jia, Lei Yang, *Graduate Student Member, IEEE*, Yan Gong, and Hongyu Pan

***Abstract*— LiDAR-based sparse 3-D object detection plays a crucial role in autonomous driving applications due to its computational efficiency advantages. Existing methods either use the features of a single central voxel as an object proxy or treat an aggregated cluster of foreground points as an object proxy. However, the former cannot aggregate contextual information, resulting in insufficient information expression in object proxies. The latter relies on multistage pipelines and auxiliary tasks, which reduce the inference speed. To maintain the efficiency of the sparse framework while fully aggregating contextual information, in this work, we propose SparseDet that designs sparse queries as object proxies. It introduces two key modules: the local multiscale feature aggregation (LMFA) module and the global feature aggregation (GFA) module, aiming to fully capture the contextual information, thereby enhancing the ability of the proxies to represent objects. The LMFA module achieves feature fusion across different scales for sparse key voxels via coordinate transformations and using nearest neighbor relationships to capture object-level details and local contextual information, whereas the GFA module uses self-attention mechanisms to selectively aggregate the features of the key voxels across the entire scene for capturing scene-level contextual information. Experiments on nuScenes and KITTI demonstrate the effectiveness of our method. Specifically, SparseDet surpasses the previous best sparse detector VoxelNeXt (a typical method using voxels as object proxies) by 2.2% mean average precision (mAP) with 13.5 frames/s on nuScenes and outperforms VoxelNeXt by 1.12% $AP_{3\text{-}D}$ on hard level tasks with 17.9 frames/s on KITTI. What is more, not only the mAP of SparseDet exceeds that of FSDV2 (a classical method using clusters of foreground points as object proxies) but also its inference speed is 1.3 times faster than FSDV2 on the nuScenes test set. The code has been released in https://github.com/liulin813/SparseDet.git.**

Received 19 June 2024; revised 1 August 2024 and 5 September 2024; accepted 20 September 2024. Date of publication 26 September 2024; date of current version 8 October 2024. This work was supported by the National Key Research and Development Program of China under Grant 2018AAA0100302. *(Corresponding author: Caiyan Jia.)*

Lin Liu, Ziying Song, Feiyang Jia, and Caiyan Jia are with the School of Computer Science and Technology, Beijing Key Laboratory of Traffic Data Analysis and Mining, Beijing Jiaotong University, Beijing 100044, China (e-mail: liulin010811@gmail.com; 22110110@bjtu.edu.cn; feiyangjia@bjtu.edu.cn; cyjia@bjtu.edu.cn).

Qiming Xia is with Fujian Key Laboratory of Sensing and Computing for Smart Cities, Xiamen University, Xiamen, Fujian 361005, China (e-mail: qimingxia96@163.com).

Lei Yang is with the State Key Laboratory of Intelligent Green Vehicle and Mobility and the School of Vehicle and Mobility, Tsinghua University, Beijing 100084, China (e-mail: yanglei20@mails.tsinghua.edu.cn).

Yan Gong is with the Autonomous Driving Department of X Division, JD Logistics, Beijing 101111, China (e-mail: gongyan2020@foxmail.com).

Hongyu Pan is with Horizon Robotics, Beijing 100190, China (e-mail: karry.pan@horizon.cc).

Digital Object Identifier 10.1109/TGRS.2024.3468394

***Index Terms*— 3-D object detection, feature aggregation, sparse detectors.**

I. Introduction

THREE-DIMENSIONAL object detection is a critical task in autonomous driving, promoting the advances of intelligent transportation systems and has gained widespread attention [1], [2], [3], [4], [5], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18]. With the availability of various sensor modalities, such as cameras and LiDAR, significant progress has been made in single-modal 3-D object detection using either camera images [19], [20], [21], [22], [23] or LiDAR point clouds [1], [1], [2]. Compared to the image data provided by cameras, LiDAR point clouds offer accurate depth and position information and have led to extensive research in recent years [24], [25], [26], [27], [28], [29], [30], [31].

Existing high-performance 3-D object detectors [25], [38], [39], [40] typically leverage 3-D sparse CNNs to extract features from sparse voxels and then convert these features into dense feature maps for detection. Although these methods [38], [39], [41] have demonstrated impressive detection accuracy on a range of benchmark datasets [32], [42], [43], [44], extending them to more practical long-range scenarios becomes challenging. This is because the computational costs associated with the dense feature mappings grow quadratically as the perception range increases. Therefore, some recent works [33], [34], [35], [36] have attempted to construct fully sparse detectors to cope with this issue, as shown in Fig. 1.

Currently, based on the strategy of object proxies, LiDAR-based sparse detectors can be categorized into two distinct classes. The first category (e.g., [35], [36]) employs stacked convolutional layers to propagate sparse features to central voxels, which are then utilized as object proxies for detection. The second category (e.g., [33], [34]) aggregates foreground points into clusters and treats them as object proxies. Although the aforementioned methods have successfully constructed fully sparse detectors, the first category only treats individual central voxels as object proxies, lacking the ability to learn point cloud context. This weakens the information representation capacity of object proxies. As shown in Fig. 2(a), treating only the central voxel features as object proxies lead to the loss of some point cloud information from the same instance. The second category clusters foreground points into object-centric clusters for

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

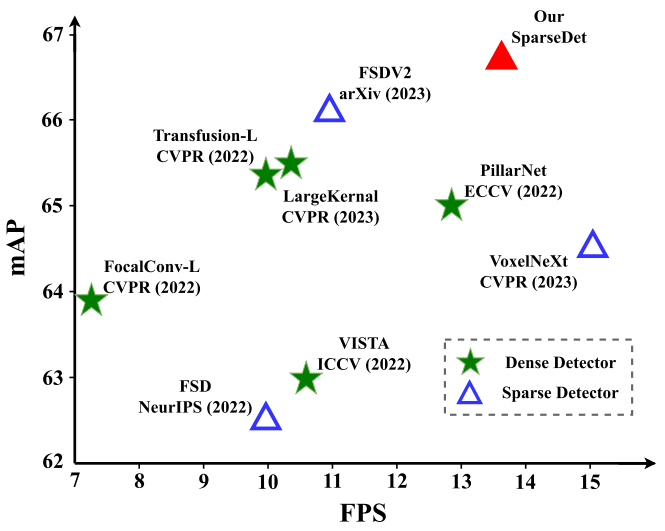


Fig. 1. Comparison of SparseDet with existing LiDAR-based detectors on the nuScenes [32] test dataset, where the vertical axis represents mAP, and the horizontal axis represents model inference speed (FPS). Compared to other sparse detectors, SparseDet achieves the highest mAP while maintaining an excellent inference speed.

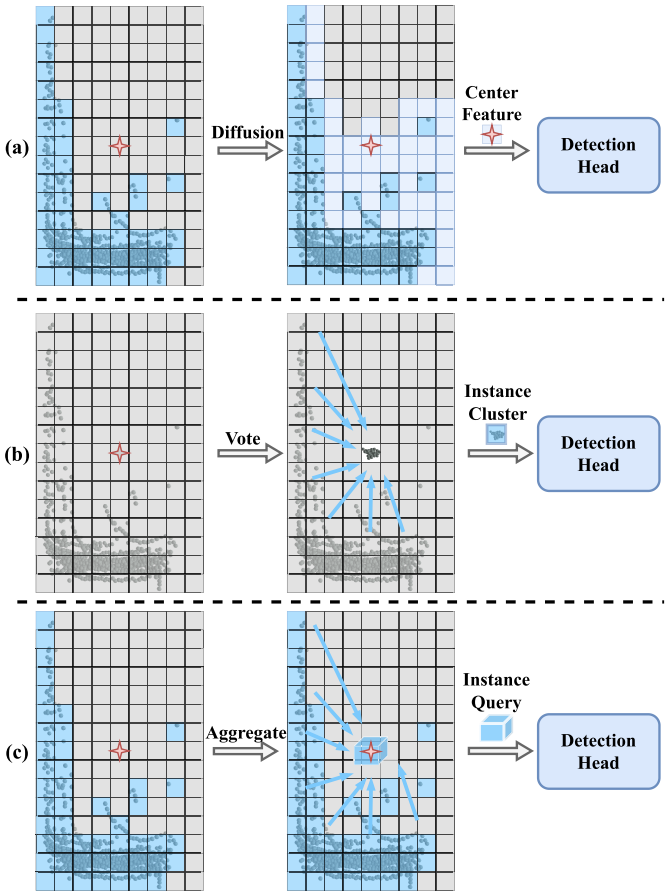


Fig. 2. **Comparison between SparseDet and other sparse detection frameworks [33], [34], [35], [36].** (a) First category methods, VoxelNeXt [35] and SWFormer [37], diffuse features by stacking convolutional layers to fill center voxels. However, utilizing only a single central voxel feature as a proxy for an object neglects a number of instance features, thereby weakening the ability to represent objects based on their central voxels. (b) Second category methods, FSD [33] and FSDV2 [34], utilize a voting mechanism to aggregate foreground points into object-centered clusters for further prediction. However, these methods overly rely on point segmentation and prediction refinement which results in time delays. (c) Our SparseDet addresses the issue of insufficient information representation in central voxel features by utilizing sparse queries as object proxies and selectively aggregating sparse voxel features at interested positions, avoiding the need for additional auxiliary tasks.

further prediction. As shown in Fig. 2(b), it (e.g., [33] or [34]) initially segments the raw point cloud into foreground and background regions, performs center voting, for instance, clustering, then extracts instance features from each cluster for initial predictions. Although these methods possess contextual aggregation capabilities, they heavily rely on additional auxiliary tasks and numerous hyperparameters, resulting in poor inference speed.

To achieve efficient detection while effectively aggregating contextual information in sparse frameworks, in this study, we propose a simple and effective fully sparse 3-D object detection framework called SparseDet. SparseDet utilizes a 3-D sparse convolutional network to extract features from point clouds and transforms them into 2-D sparse features for further prediction via the detection head. As shown in Fig. 2(c), SparseDet designs sparse queries as object proxies, allowing to flexibly and selectively aggregate point clouds to obtain object proxies in a scene. Compared to the previous sparse aggregation paradigm [33], [34], first, SparseDet extends the aggregation of local contextual information to multiscale feature space, thereby obtaining richer local information. Furthermore, in contrast to prior methods [33], [34] that only focus on aggregating foreground point features, SparseDet can aggregate the scene-level context for each instance to facilitate potential collaboration between the scene and instance features. Finally, SparseDet does not require any additional auxiliary tasks.

To enhance SparseDet's ability to learn point cloud context, we have designed two key modules: the local multiscale feature aggregation (LMFA) module and the global feature aggregation (GFA) module. The LMFA module is aimed to capture local contextual information at multiple scales by only leveraging simple coordinate transformations and voxel's nearest-neighbor relationships to collect features from neighboring key voxels. These features are then aggregated to obtain richer local representations. Through these operations, the LMFA module enables SparseDet to capture fine-grained details and local variations in point clouds. This enhances the expressive power of sparse voxel features and enables a better representation of the underlying structures within point clouds. Afterward, we initialize the aggregated features of the key voxels as sparse queries and feed them into the GFA module. GFA module targets to aggregate global point cloud information across the entire scene by utilizing global sparse queries in a larger receptive field. This allows SparseDet to have a comprehensive understanding of the scene and incorporates global context into the detection process. Benefiting from the comprehensive learning of contextual information, SparseDet significantly enhances the information representation capability of object proxies and demonstrates superior performance (see Fig. 1 as a reference).

In summary, the main contributions of our work are as follows.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

1) We have summarized the issues present in existing sparse detection methods and proposed SparseDet to address these problems, which enables to improve the performance of sparse Lidar-based 3-D object detection.
2) SparseDet is composed of two modules—the LMFA module and the GFA module—to achieve efficient detection by effectively aggregating contextual information in a sparse framework.
3) Experiments conducted on the KITTI [42] and nuScenes [32] benchmarks demonstrate the effectiveness of our SparseDet. On the nuScenes dataset, SparseDet achieves better performance than FSDV2 and is 1.38 times faster than FSDV2 on frames per second (FPS). And on KITTI, SparseDet surpasses VoxelNeXt by 1.12% AP_{3-D} on hard-level tasks.

II. Related Work

A. LiDAR-Based Dense Detectors

Although point cloud data exhibits different sparsity characteristics compared to 2-D image data, 3-D object detectors [25], [26], [38], [45], [46], [47] are often designed by referencing 2-D detectors. Despite differences in the design of backbone networks in these works, for example, methods [26], [45] represented by VoxelNet use dense 3-D backbone networks, whereas methods like SECOND [25], [38], [46], [47] use sparse 3-D backbone networks. However, they all employ dense 2-D detection heads to address the 3-D detection problem. These methods are generally referred to as LiDAR-based dense detectors.

As a pioneer, VoxelNet [26] partitions point clouds into regular grids and utilizes a 3-D backbone network for feature extraction. It then applies a dense head for prediction. Based on VoxelNet, SECOND [25] implements efficient computation of sparse convolution and submanifold convolution operators to gain fast inference speed by constructing a hash table. However, SECOND still requires dense bird's eye view (BEV) feature maps and a dense detection head for detection. On the influence of SECOND, most subsequent networks [38], [45], [46], [48], [49], [50], [51] follow the paradigm of utilizing a 3-D sparse backbone combined with a 2-D dense detection head.

Although LiDAR-based dense detectors have shown excellent performance on multiple benchmark datasets [32], [42], [43], [44], their reliance on dense BEV feature maps and dense detection heads makes them challenging to scale-up to long-range scene detection. This is because the computational costs of dense BEV feature maps increase quadratically with detection distance [33]. This drawback significantly restricts the practical applications of LiDAR-based dense detectors in real-world scenarios.

B. LiDAR-Based Sparse Detectors

Currently, sparse detectors include point-based methods and partial voxel-based methods. Point-based methods [27], [28], [52], [53] utilize key points within point clouds for feature aggregation and detection. These methods do not require dense sampling and computation across the entire space, making them inherently sparse detectors. FSD [33] and FSDV2 [34] are representative of this series of methods. FSD represents individual objects by clustering segmented foreground points. It then fed features that are extracted by PointNet into a detection head for calibration and prediction. In FSDv2, the instance clustering step is replaced with a virtual voxelization module, which aims to remove the inherent bias introduced by manually crafted instance-level representations. Despite adequately aggregating foreground information, the reliance on additional auxiliary tasks and numerous hyperparameters leads to poor inference speed.

Among the voxel-based sparse methods, VoxelNeXt [35] introduces additional downsampling layers to place voxels near the centers of objects and subsequently performs feature diffusion on key voxels to propagate features toward the object centers. Although VoxelNeXt [35] has achieved impressive efficiency, it solely relies on single center voxel features for detection, which significantly weakens the information representation capability of object proxies, ultimately leading to a decline in model performance. As mentioned before, treating only the central voxel features as object proxies leads to the loss of some point cloud information from the same instance as illustrated in Fig. 2(a). In this work, we use sparse queries and attention mechanisms to obtain object proxies by LMFA and GFA modules, which enable us to dynamically capture contextual information at different granularities. This promotes the collaboration between scene-level and instance-level features, thereby enabling the model to obtain richer and more accurate object representations.

III. SparseDet

In this section, we propose a simple and efficient LiDAR-based sparse detection framework SparseDet. Fig. 3 illustrates its structure which follows the pipelines of the fully sparse network VoxelNeXt [35]. But differently, to fully aggregate the contextual information in point clouds to enhance the information expression capability of the sparse object proxies, we design two modules—the LMFA module and the GFA module. The two modules intend to adaptively aggregate multilevel contextual information on point clouds and make SparseDet able to strongly enhance the information representation capability of object proxies to improve the performance of 3-D detection with low computational costs.

A. Local Multiscale Feature Aggregation

Most LiDAR-based sparse detection methods [33], [34], [35], [36] utilize center voxel features as object proxies for detection. Although using center features as object proxies provides accurate positional information, a single-center voxel feature is insufficient to fully capture the entire information of an object. This severely weakens the expressive capability of the object proxies. Therefore, we propose an LMFA module to compensate for the shortcomings. In the LMFA module, we focus on learning the local contextual information around an object, which helps to understand details such as the shape, size, and relative position of a target object. As shown in Fig. 3(a), we dynamically aggregate neighborhood information

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

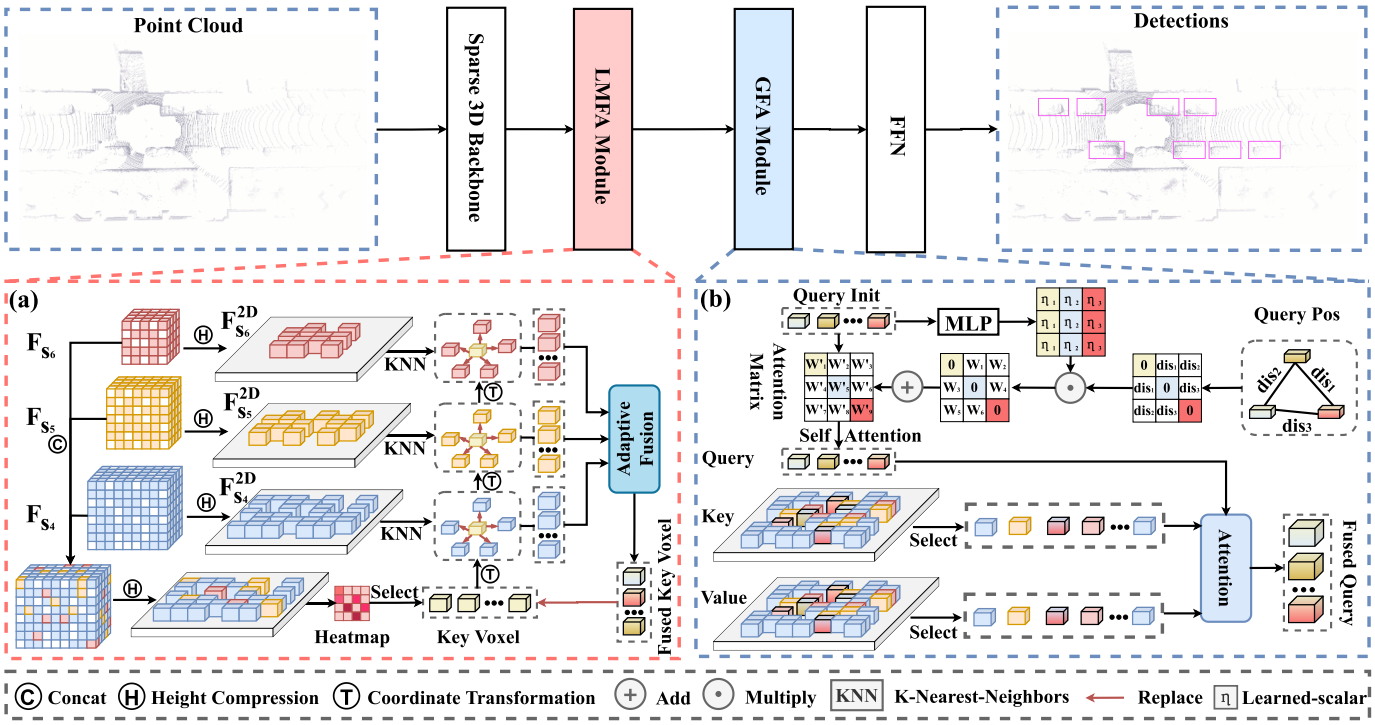


Fig. 3. Framework of our **SparseDet**. First, we voxelize point clouds and feed voxels into a 3-D sparse convolution backbone. (a) In the LMFA module, we perform high compression [35] on sparse voxel features (F_{s_4}, F_{s_5}, and F_{s_6}) of the last three layers in the 3-D backbone to obtain 2-D sparse features of these three layers, denoted as $F_{s_4}^{2D}$, $F_{s_5}^{2D}$, and $F_{s_6}^{2D}$. In LMFA, we concat F_{s_4}, F_{s_5}, and F_{s_6} to obtain F_{Fusion}. After applying high compression [35] on F_{Fusion}, we have F_{Fusion}^{2D} and then perform key voxel position prediction using a heatmap. Through coordinate transformation, we convert the key voxel features to the spaces of $F_{s_4}^{2D}$, $F_{s_5}^{2D}$, and $F_{s_6}^{2D}$ and aggregate the neighborhood voxel context based on K nearest-neighbor (KNN) relationships. Subsequently, we replace the key voxel features by aggregating voxel features back into F_{Fusion}^{2D} based on the indices to enhance the feature representation capability of sparse features. (b) In the GFA module, we utilize sparse voxels as queries to adaptively aggregate global sparse voxel features, where a scale-adaptive weight map allows queries to autonomously learn the receptive field for aggregating information from relevant positions. At last, the aggregated queries are fed into FFN for result prediction. Adaption fusion means the adaptive fusion of multiscale features and FFN is a feed-forward neural network.

of key voxels through KNN positional relations to enhance their feature representation capability. The aggregated key voxel features will then be used to initialize the sparse object queries. It is worth noting that considering the distribution differences in 3-D object scales, we extend LMFA to multiple-scale spaces. Thus, LMFA primarily consists of two steps—**Sparse Key Voxel Selection** and **Fusion of Voxel Features from Different Scales**.

1) Sparse Key Voxel Selection: First, we voxelize point clouds and feed them into a 3-D sparse convolutional backbone network. Referring to VoxelNeXt [35], we add two additional downsampling layers to the 3-D sparse backbone network [25]. This step serves two key purposes. First, it constructs multiscale feature spaces through the additional downsampling process to facilitate the subsequent feature aggregation in the LMFA module. Second, through the additional sampling and height compression operations, we can place voxel features in object centers that are blank to construct neighborhood relations more accurately. With the above operations, the original sparse 3-D convolutional backbone transitions from $\{F_{s_1}, F_{s_2}, F_{s_3}, F_{s_4}\}$ to $\{F_{s_1}, F_{s_2}, F_{s_3}, F_{s_4}, F_{s_5}, F_{s_6}\}$ with feature strides $\{1, 2, 4, 8, 16, 32\}$, where $F_{s_i} \in R^{N_i \times C_i}$ represents the 3-D sparse voxel features for each stage, s_i means the ith stage, N_i is the number of nonempty voxels, and C_i is the number of channels. Afterward, we transform F_{s_5} and F_{s_6} to the feature space of F_{s_4} and concatenate F_{s_4}, F_{s_5}, and F_{s_6} together to obtain F_{Fusion}. Then, we perform high compression on F_{Fusion}, F_{s_4}, F_{s_5}, and F_{s_6} to obtain F_{Fusion}^{2D}, $F_{s_4}^{2D}$, $F_{s_5}^{2D}$, and $F_{s_6}^{2D}$. Specifically, following the VoxelNeXt [35], we replace all the voxel features on the ground plane and sum them up at the same positions.

To select key voxels, we use heatmap operation that predicts voxel scores Score $\in R^{N_{\text{voxel}} \times \text{Cls}}$ for Cls classes based on the sparse voxel feature F_{Fusion}^{2D}, where N_{voxel} represents the number of nonempty voxels in F_{Fusion}^{2D}. We designate the voxels closest to an object center as positive samples and utilize FocalLoss [38] for supervision. This means that voxels with higher scores have a higher probability of belonging to the foreground. Subsequently, we apply the top-N_{key} score operation to F_{Fusion}^{2D} to obtain N_{key} key sparse voxel candidates. Here, N_{key} is set to a default value of 500. The workflow is demonstrated in Algorithm 1.

2) Fusion of Voxel Features From Different Scales: In this section, we construct a KNN graph to obtain neighborhood information for sparse candidate voxels at different scales to gain more comprehensive local contexts that address the insufficient information representation capacity of sparse features.

After sparse key voxel selection, we obtain the features of N_{key} sparse voxels, denoted as $F_{\text{key}} \in R^{N_{\text{key}} \times C}$, where C represents the number of channels. The corresponding coordinate position indices are defined as I_{key}, with a shape of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Sparse Key Voxel Selection Workflow

Input: The sparse voxel features from multiple stages: $F_{s_i} \in R^{N_i \times C_i}, i = 4, 5, 6$. The spatial coordinates of each sparse voxel in 3-D space: $I_{s_i} \in R^{N_i \times 3}, i = 4, 5, 6$.

Output: The features of selected key voxels: $F_{key} \in R^{N_{key} \times C}$

$I_{s_5} = I_{s_5} \times 2, I_{s_6} = I_{s_6} \times 4$
$I_{Fusion} = cat(I_{s_4}, I_{s_5}, I_{s_6})$
$F_{Fusion} = cat(F_{s_4}, F_{s_5}, F_{s_6})$
$i_{unique}, inv = torch.unique(I_{Fusion})$
$f_{unique} = torch.zeros(i_{unique}.shape[0], F_{Fusion}.shape[1])$
$f_{unique} = f_{unique}.index_add(inv, F_{Fusion})$
$f_{out} = spconv.SparseTensor($
    $features = f_{unique},$
    $indices = i_{unique})$
$Score = Head(f_{out})$
$top_key = Score.argsort()[\ldots, : N_{key}]$
$top_key_index = top_key\%Cls$
$F_{key} = f_{out}.features.gather($
    $index = top_key_index,$
    $dim = -1)$
Return F_{key}

$(N_{\text{key}}, 2)$, which represents the 2-D position indices. We first divide the position coordinates of the N_{key} voxels in the S_4 scale, denoted as I_{s_4}, by 2 and 4, respectively, to transform them into the lower-resolution voxel spaces of $\{S_5, S_6\}$. We then save the corresponding spatial coordinate indices as I_{s_5}, I_{s_6}. Given the position coordinate information of N_{key} sparse voxels in different scale spaces, we aim to find the K nearest voxels $N^{s_i}_{\text{Neighbor}}$ for each key voxel. The value of $N^{s_i}_{\text{Neighbor}}$ is halved as the scale space changes, which can be determined using the following formula:

$$N^{s_i}_{\text{Neighbor}} = M/2^{i-4}, \quad i = 4, 5, 6 \tag{1}$$

where the parameter M is set to 8 by default.

To improve the efficiency of LMFA, we employ the *KD-Tree* algorithm to obtain the indices of $N^{s_i}_{\text{Neighbor}}$ neighbors for each key voxel at a specific scale S_i, which are denoted as $I^{s_i}_{\text{Neighbor}} \in R^{N_{\text{key}} \times N^{s_i}_{\text{Neighbor}} \times 2}$. The surrounding neighbor voxel features $F^{s_i}_{\text{Neighbor}} \in R^{N_{\text{key}} \times N^{s_i}_{\text{Neighbor}} \times C}$ are obtained by indexing $F^{\text{2D}}_{s_i}$ with $I^{s_i}_{\text{Neighbor}}$. Then, Conv_{1*1} is utilized to aggregate the features of the neighbor voxel features $F^{s_i}_{\text{Neighbor}}$, which is performed by the following formula:

$$F^{s_i}_{\text{Neighbor}'} = \text{Conv}_{1*1}\left(F^{s_i}_{\text{Neighbor}}\right) \tag{2}$$

where $F^{s_i}_{\text{Neighbor}'} \in R^{N_{\text{key}} \times 1 \times C}$ represents aggregated features. By applying the aforementioned feature aggregation scheme at each feature scale in $\{S_4, S_5, S_6\}$, we can obtain aggregated features of multiple scales $F_{\text{Neighbor}} \in R^{N_{\text{key}} \times N_{\text{scale}} \times C}$, where N_{scale} is the number of scales.

Given the encoded multiscale features of sparse voxels, a naive fusion approach is to concatenate the multiscale features to form a single feature [45]. However, we observe that some object detections rely more on information from specific scales rather than from all scales. For instance, low-resolution feature maps lack information about small objects. Therefore, key voxels related to small objects should more effectively gather information solely from high-resolution feature maps. We propose utilizing learnable scale weights to automatically select the scale for each key voxel F_{key} as follows:

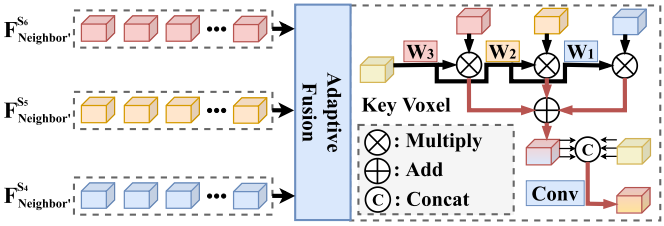


Fig. 4. Architecture of **adaptive fusion**, which adaptively weights and fuses neighborhood features ($F^{s_4}_{\text{Neighbor}'}$, $F^{s_5}_{\text{Neighbor}'}$, and $F^{s_6}_{\text{Neighbor}'}$) from multiscale feature spaces for each key voxel.

$$W_1, W_2, W_3 = \text{Softmax}(\text{FC}(F_{\text{key}}))$$
$$F_{\text{key}} = \text{Conv}\left(\text{Concat}\left(F_{\text{key}}, \sum_{i=1}^{N_{\text{scale}}} W_i F^{s_i}_{\text{neighbor}'}\right)\right) \tag{3}$$

where W_1, W_2, and W_3 stand for the importance of selecting $F^{s_4}_{\text{neighbor}'}$, $F^{s_5}_{\text{neighbor}'}$, $F^{s_6}_{\text{neighbor}'}$, and F_{key} is adaptively aggregated according to the scaled attention weights W_i obtained in (3). These scale weights can be learned during the training process and enable adaptive scale selection based on the characteristics of individual key voxels. Thus, we also call the step adaptive fusion.

With such a scale-selection mechanism, the scale most related to each key voxel is softly selected while the visual features from other scales are suppressed. We then place F_{key} into $F^{\text{2D}}_{\text{Fusion}}$ according to the position index of F_{key}, obtaining the enhanced $F^{\text{2D}'}_{\text{Fusion}} \in R^{N_{\text{voxels}} \times C}$. The whole process of our adaptive fusion is illustrated in Fig. 4.

B. Global Feature Aggregation

The LMFA module aims to learn the local contextual information around objects by dynamically aggregating neighborhood information of key voxels through the use of nearest-neighbor positional relations.

Although the fusion of neighborhood voxel features enhances the expressive power of foreground sparse voxel features, the LMFA module still has limitations when dealing with sparse detection scenarios: 1) for a large object, using a single aggregated sparse voxel as a proxy for object detection still suffers from information loss since the object proxy should contain information from the entire object rather than just a local region and 2) LMFA ignores the potential collaboration between the whole scene and instance features. For example, a false negative object in a scene can be potentially rectified by enhancing its feature through interactions with instances sharing similar semantic information. Therefore, we propose the GFA module to further address the limitations of the LMFA module by learning the global structural and semantic information of the entire

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

scene. This makes SparseDet able to leverage the contextual information of objects in both local and global ways to eliminate ambiguity, thereby improving detection accuracy.

In detail, after the LMFA module, we have the aggregated features of the key voxels $F_{\text{key}} \in R^{N_{\text{key}} \times C}$ and the enhanced voxel features of the entire scene $F_{\text{Fusion}}^{\text{2D}'} \in R^{N_{\text{voxels}} \times C}$, where N_{voxels} is the number of nonempty voxels. Afterward, we select the top-N_{query} highest-scoring voxels based on Score $\in R^{N_{\text{voxels}} \times \text{Cls}}$ to initialize object queries. N_{query} is set to 200 by default. We use the selected voxel positions $\text{Pos}_{\text{query}}$ from $\mathbb{R}^{N_{\text{query}} \times 2}$ and the features F_{query} from $\mathbb{R}^{N_{\text{query}} \times C}$ to initialize the position encoding and the queries Q, as shown in (4). In this way, the initial object queries are located at or near potential object centers and contain more object information, enabling the queries to learn more efficiently and eliminating the need for multiple decoder layers to refine positions in query-based methods [54], [55]

$$\begin{aligned} Q &= F_{\text{query}} + \text{PE}_{\text{query}} \\ \text{PE}_{\text{query}} &= \text{MLP}(\text{MLP}(\text{Pos}_{\text{query}})) \end{aligned} \tag{4}$$

where PE_{query} means the position embedding of $\text{Pos}_{\text{query}}$. After obtaining the top-N_{query} queries, we use the sparse voxel features $F_{\text{Fusion}}^{\text{2D}}$ as Key and Value in self-attention to obtain the attention score for each of top-N_{query} queries to perform feature aggregation. However, due to the varying number of nonempty voxels in different scenes, traditional decoders cannot be directly applied to the GFA module. Thus, we have conducted data analysis and found that the number of nonempty voxels in a single scene ranges approximately from 9k to 13k. We then select a fixed number of $N_{K,V}$ sparse voxel features as the Key and the Value from the sparse voxel features $F_{\text{Fusion}}^{\text{2D}'}$ based on Score $\in R^{N_{\text{voxel}} \times \text{Cls}}$. This allows SparseDet to focus more on foreground features. Here, $N_{K,V}$ is set to a default value of 10K, and any shortfall in the number of selected features is padded with zeros.

In most approaches for voxel selection [35], [54], it is common to flatten and sort Score $\in R^{N_{\text{voxel}} \times \text{Cls}}$ along the channel dimension. However, the aforementioned strategy cannot be directly applied in the GFA module. The reason is that individual voxels often have similar scores for multiple classes, such as pedestrian and traffic cones. Relying solely on class scores as the selection criterion would result in the problem of repetitive feature selection. To address this issue, we propose a solution where we do not differentiate the scores for different classes. Instead, we consider the highest score within each voxel's class as the probability of that voxel being a foreground voxel. Specifically, we apply max-pooling along the channel dimension of Score $\in R^{N_{\text{voxel}} \times \text{Cls}}$ to obtain $\text{Score}' \in R^{N_{\text{voxel}} \times 1}$, which represents the highest score for each voxel regardless of its class. Next, we select $N_{K,V}$ sparse voxel features based on Score$'$ as the Key and the Value denoted as $F_K \in R^{N_{K,V} \times C}$ and $F_V \in R^{N_{K,V} \times C}$.

To highlight the importance of target objects while maintaining a global perspective, the GFA module should interact more with the voxel features near the queries. Inspired from [56], we introduce scale-adaptive self-attention (SASA) to GFA, allowing it to learn an appropriate receptive field guided by the queries. First, SASA computes the pairwise distances $\text{Dis} \in R^{N_{\text{query}} \times N_{\text{query}}}$ between all pairs of query centers $\text{Pos}_{\text{query}}$ as

$$\text{Dis}_{i,j} = \sqrt{\left(X_{\text{Pos}}^i - X_{\text{Pos}}^j\right)^2 + \left(Y_{\text{Pos}}^i - Y_{\text{Pos}}^j\right)^2} \tag{5}$$

where $(X_{\text{Pos}}^i, Y_{\text{Pos}}^i)$ denotes the position of the ith query. Consequently, updated queries can be obtained in the following:

$$Q = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + \eta \log(\text{Dis})\right) V \tag{6}$$

where η controls the receptive field of each query and is generated through a linear transformation. In this way, the queries Q can dynamically adjust the size of receptive fields and prioritize features in the vicinity of the query centers. The workflow of the GFA module is detailed in Algorithm 2. Afterward, the aggregated and refined queries Q are passed through a feed-forward network (FFN) layer for object prediction.

Algorithm 2 The GFA Workflow

Input:
The height compressed sparse voxel features: $F_{Fusion}^{2D'}$
The heatmap of sparse voxels: $Score \in R^{N_{voxels} \times Cls}$
Output: The object queries: **Q**

```
def SASA(Q, Pos_query):
    Dis = Calculate_dis(Pos_query)
    eta = MLP(Q)
    attn_mask = Dis * eta
    Q = self_attention(Q, attn_mask)
    Return Q
if use GFA then
    top_pro = Score.argsort()[..., : N_query]
    top_pro_cls = top_pro//N_voxels
    top_pro_index = top_pro%Cls
    Q_feat = F_Fusion^2D'.features.gather(
                index = top_pro_index,
                dim = −1)
    Pos_query = F_Fusion^2D'.indices.gather(
                index = top_pro_index,
                dim = −1)
    Score' = Max_pool(Score)
    top_kv = Score'.argsort()[..., : N_K,V]
    K, V_feat = F_Fusion^2D'.features.gather(
                index = top_kv,
                dim = −1)
    Pos_K,V = F_Fusion^2D'.indices.gather(
                index = top_kv,
                dim = −1)
    Q = SASA(Q, Pos_query)
    Q = decoder(Q_feat, K, V_feat, Pos_query, Pos_K,V)
    Return Q
end
```

IV. Experiments

In this section, we present the details of each dataset and the experimental setup of SparseDet and evaluate SparseDet's

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON WITH THE SOTA METHODS ON NUSCENES *Test* SET. "C.V.", "MOTOR.", "PED.", AND "T.C." ARE SHORT FOR CONSTRUCTION VEHICLE, MOTORCYCLE, PEDESTRIAN, AND TRAFFIC CONE, RESPECTIVELY. "SPARSE" REFERS TO METHODS THAT USE SPARSE BACKBONE WITH SPARSE DETECTION HEAD. "DENSE" REFERS TO METHODS THAT USE DENSE DETECTION HEAD

	Method	mAP	NDS	Car	Truck	C.V.	Bus	Trailer	Barrier	Motor.	Bike	Ped.	T.C.
Dense	PointPillars [24]	30.5	45.3	68.4	23.0	4.1	28.2	23.4	38.9	27.4	1.1	59.7	30.8
	CenterPoint [38]	58.0	65.5	84.6	51.0	17.5	60.2	53.2	70.9	53.7	28.7	83.4	76.7
	HotSpotNet [57]	59.3	66.0	83.1	50.9	23.0	56.4	53.3	71.6	63.5	36.6	81.3	73.0
	InfoFocus [58]	39.5	39.5	77.9	31.4	10.7	44.8	37.3	47.8	29.0	6.1	63.4	46.5
	AFDetV2 [50]	62.4	68.5	86.3	54.2	26.7	62.5	58.9	71.0	63.8	34.3	85.8	80.1
	VISTA [59]	63.0	69.8	84.4	55.1	25.1	63.7	54.2	71.4	70.0	45.4	82.8	78.5
	Focals Conv [51]	63.8	70.0	86.7	56.3	23.8	67.7	59.5	74.1	64.5	36.3	81.4	81.4
	TransFusion-L [54]	65.5	70.2	86.2	56.7	28.2	66.3	58.8	78.2	68.3	44.2	86.1	82.0
	UVTR [60]	67.1	71.1	87.5	56.0	33.8	67.5	59.5	73.0	73.4	54.8	86.3	79.6
	LargeKernel3D [61]	65.3	70.5	85.9	55.3	26.8	65.7	62.1	75.5	72.5	46.6	85.6	80.0
	PVT-SSD [62]	53.6	65.0	79.4	43.8	21.7	62.1	34.2	67.1	53.4	38.2	79.8	56.6
Sparse	3DSSD [63]	42.7	56.4	81.2	47.2	12.6	61.4	30.5	47.9	36.0	8.6	70.2	31.1
	SASA [64]	45.0	61.0	76.8	45.0	16.1	66.2	36.5	53.6	39.6	16.9	69.1	29.9
	FSD [33]	62.5	68.7	-	-	-	-	-	-	-	-	-	-
	FSDV2 [34]	66.2	71.7	83.7	51.6	**32.5**	66.4	**59.1**	**78.7**	71.4	**51.7**	87.1	80.3
	VoxelNeXt [35]	64.5	70.0	84.6	53.0	28.7	64.7	55.8	74.6	73.2	45.7	85.8	79.0
	SparseDet	**66.7**+2.2	**71.9**+1.9	**86.2**+1.6	**56.0**+3.0	30.2+1.5	**66.5**+1.8	58.4+2.6	78.7+4.1	**73.7**+0.5	46.8+1.1	**87.5**+1.7	**82.5**+3.5

performance of 3-D object detection on KITTI [42] and nuScenes [32] datasets.

A. Dataset and Evaluation Metrics

1) nuScenes Dataset: The nuScenes dataset [32] is a large-scale 3-D detection benchmark consisting of 700 training scenes, 150 validation scenes, and 150 testing scenes. The dataset was collected using six multiview cameras and a 32-beam LiDAR sensor. It includes 360° object annotations for 10 object classes. To evaluate the detection performance, the primary metrics used are the mean average precision (mAP) and the nuScenes detection score (NDS) [32], which assesses detection accuracy in terms of classification, bounding box location, size, orientation, attributes, and velocity.

2) KITTI Dataset: The KITTI dataset [42] provides synchronized LiDAR point clouds and front-view camera images. It consists of 7481 training samples and 7518 test samples. As a common practice [46], [48], [51], the training data are divided into a train set with 3712 samples and a validation set (*val* set for short) with 3769 samples to conduct the evaluation. To evaluate the test dataset using the official KITTI test server, we follow the approach outlined in Voxel RCNN [46], using 80% of the 7481 training samples to train our model SparseDet with the amount of 5985 samples. The standard evaluation metric for object detection is the mAP, computed using recall at 40 positions (R40). In this work, we evaluate SparseDet and all other dense and sparse methods on the most commonly used classes including Car, Pedestrian, and Cyclist using average precision (AP) with an intersection over union (IoU) threshold of 0.7, 0.5, and 0.5, respectively.

B. Implementation Details

In this section, we will provide a detailed description of the SparseDet settings for KITTI [42] and nuScenes [32]. To enable effective training on KITTI and nuScenes, we utilize four NVIDIA RTX 3090 24G GPUs with Intel[1] Xeon Platinum 8358P CPU@2.60 GHz. During inference, the batch size is set to 1 on a 3090 GPU. All latency measurements are taken on the same workstation with a 3090 GPU. We implement SparseDet based on the open-source code repositories OpenPCDet [77] and the baseline VoxelNeXt [35] using CUDA version 11.3 and the deep learning framework PyTorch version 1.12.1 + cu113.

For KITTI [42], the input voxel size is set to [0.05, 0.05, 0.1 m] and the point range is set to [0, −40, −3, 70.4, 40, 1 m] across the X-, Y-, and Z-axes, respectively. The maximum number of point clouds contained in each voxel is set to 10. During training, following the baseline [35], SparseDet is trained 80 epochs with a batch size of four per GPU for 3 h (total train time).

For nuScenes [32], the input voxel size is set to [0.075, 0.075, 0.2 m] and the point range is set to [−54, −54, −5, 54, 54, 3 m] across the X-, Y-, and Z-axes, respectively. During training, following the baseline [35], SparseDet is trained 20 epochs with a batch size of two per GPU for 46 h (total train time). For more details concerning our method, refer to OpenPCDet [77].

C. Comparison With State-of-the-Arts

1) Performance on the nuScenes Test Dataset: We conduct experiments on the large-scale nuScenes [32] dataset using the SOTA (state-of-the-art) fully Sparse 3-D detector VoxelNeXt [35] (a typical method of the second category mentioned in related works) as the baseline to further validate the effectiveness of our SparseDet, as shown in Table I. In the table, we not only compare our SparseDet with classical sparse detectors of the first category including FSD [33] and FSDV2 [34] but also include some SOTA dense 3-D detectors such as UVTR [60], Transfusion-L [54], LargeKernel3D [61], and PVT-SSD [62] for comparison.

Based on VoxelNeXt [35], our SparseDet achieves 66.7% mAP and 71.3% NDS, which surpasses the baseline by 2.2% mAP and 1.3% NDS. It is worth noting that our method achieves state-of-the-art AP in most categories. Specifically, for the categories of "Truck", "Barrier", and "T.C.", our

[1]Registered trademark.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON WITH THE BASELINE ON NUSCENES *val* DATASET. "C.V.", "PED.", AND "T.C." ARE SHORT FOR CONSTRUCTION VEHICLE, PEDESTRIAN, AND TRAFFIC CONE, RESPECTIVELY

Dataset Split	Method	mAP	NDS	Car	Truck	C.V.	Bus	Trailer	Barrier	Motor.	Bike	Ped.	T.C.
full	VoxelNeXt*	60.8	68.1	83.8	57.7	20.8	71.5	38.6	67.6	63.0	51.5	84.5	69.0
	SparseDet	65.3+4.5	70.3+2.2	87.5+3.7	60.2+2.5	27.2+6.4	75.8+4.3	40.4+1.8	73.1+5.5	69.7+6.7	58.8+7.3	86.5+2.0	63.4+4.4
$\frac{1}{4}$	VoxelNeXt*	51.8	60.7	80.2	48.1	15.4	63.1	26.1	59.3	52.5	35.5	81.6	56.2
	SparseDet	55.3+3.5	62.7+2.0	82.7+2.5	51.8+3.7	18.8+3.4	64.3+1.2	27.6+1.5	64.6 +5.3	60.1+7.6	39.1+3.6	81.6	62.7+6.5
$\frac{1}{8}$	VoxelNeXt*	46.6	55.4	78.0	41.8	15.9	57.7	20.6	53.1	49.4	23.7	76.3	49.4
	SparseDet	49.5+2.9	58.2+2.8	81.3+3.3	44.8+3.0	17.4+1.5	63.2+5.5	24.1+3.5	54.2+1.1	54.2+4.8	25.7+2.0	78.8+2.5	52.6+3.2

* denotes re-implement result.
The color red indicates improvement.

TABLE III

PERFORMANCE COMPARISON WITH SOTA METHODS ON THE KITTI *Test* SET FOR 3-D DETECTION WITH AN AP OF 40 SAMPLING RECALL POINTS EVALUATED ON THE KITTI SERVER. "SPARSE" REFERS TO METHODS THAT USE SPARSE BACKBONE WITH SPARSE DETECTION HEAD. "DENSE" REFERS TO METHODS THAT USE DENSE DETECTION HEAD

	Method	Car						Pedestrian						Cyclist					
		AP_{3D} (%)			AP_{BEV} (%)			AP_{3D} (%)			AP_{BEV} (%)			AP_{3D} (%)			AP_{BEV} (%)		
		Easy	Mod	Hard	Easy	Mod	Hard	Easy	Mod	Hard	Easy	Mod	Hard	Easy	Mod	Hard	Easy	Mod	Hard
Dense	BSAODet [65]	88.89	81.74	77.14	-	-	-	51.71	43.63	41.09	-	-	-	**82.65**	**67.79**	60.26	-	-	-
	$H^2$3D R-CNN [66]	90.43	81.55	77.22	92.85	88.87	86.07	52.75	45.26	41.56	58.14	50.43	46.72	78.67	62.74	55.78	82.76	67.90	60.49
	SECOND [25]	84.65	75.96	68.71	91.81	86.37	81.04	-	-	-	-	-	-	-	-	-	-	-	-
	PointPillars [24]	82.58	74.31	68.99	90.07	86.56	82.81	51.45	41.92	38.89	57.60	48.64	45.78	77.10	58.65	51.92	79.90	62.73	55.58
	SIEV-Net [67]	85.21	76.18	70.60	-	-	-	54.00	44.80	41.11	-	-	-	78.75	59.99	52.37	-	-	-
	TANet [68]	83.81	75.38	67.66	-	-	-	**54.92**	46.67	**42.42**	-	-	-	73.84	59.86	53.46	-	-	-
	Voxel RCNN [46]	**90.90**	81.62	77.06	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	Voxel RCNN *	90.76	81.69	77.42	92.89	89.97	84.69	52.57	44.86	39.09	57.66	49.32	44.15	77.54	64.00	53.15	79.68	67.56	62.70
	PV-RCNN [48]	90.25	81.43	76.82	94.98	90.65	86.14	52.17	43.29	40.29	59.86	50.57	46.74	78.60	63.71	57.65	82.49	68.89	62.41
	PV-RCNN *	90.61	81.51	76.81	94.68	90.87	86.19	52.10	43.63	40.44	60.06	50.43	46.81	78.58	63.83	57.71	82.50	68.93	62.57
	PV-RCNN++ * [69]	87.72	81.29	76.78	91.93	89.75	85.77	47.50	40.31	38.15	51.20	44.61	42.55	80.34	67.46	60.38	83.83	71.51	64.77
	PVT-SSD [62]	90.65	**82.29**	76.85	**95.23**	**91.63**	86.43	-	-	-	-	-	-	-	-	-	-	-	-
Sparse	VoxSet [70]	88.53	82.06	77.46	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	Part-A^2 [49]	87.81	78.49	73.51	91.70	87.79	84.61	53.10	43.35	40.06	59.04	49.81	45.92	79.17	63.52	56.93	**83.43**	68.73	61.85
	VoxelNeXt * [35]	89.10	80.35	76.99	93.03	89.49	86.40	52.10	42.72	39.08	57.19	48.27	44.55	81.33	65.31	57.43	83.34	67.55	59.61
	SparseDet	90.79	81.17	**78.11**	94.48	89.68	**87.53**	52.92	44.61	41.80	59.84	50.75	46.57	81.93	65.95	**60.41**	83.16	**69.04**	**63.46**
	improvement	*+1.69*	*+0.82*	*+1.12*	*+1.45*	*+0.19*	*+1.13*	*+0.82*	*+1.89*	*+2.72*	*+2.65*	*+2.48*	*+2.02*	*+0.60*	*+0.64*	*+2.98*	*-0.18*	*+1.49*	*+3.85*

* denotes re-implement result.

method shows significant improvements with AP gains of 3.0%, 2.6%, and 3.5%, respectively. Overall, the above results fully demonstrate the generalization and effectiveness of our method. Thanks to our LMFA and GFA modules, SparseDet can obtain richer and more accurate object representations by comprehensively aggregating local point cloud details and global contextual information, while also facilitating the collaboration between the entire scene and instance features. As a result, it achieves significant performance improvements across a wide range of object categories.

2) Performance on the nuScenes val Dataset: To demonstrate the effectiveness of our SparseDet, we conduct experiments on the nuScenes *val* dataset using VoxelNeXt [35] as the baseline. As shown in Table II, our method outperforms VoxelNeXt by 6.4%, 6.7%, and 7.3% on "C.V.", "Motor", and "Bike" categories in the nuScenes full *val* dataset, respectively. The results demonstrate that the effective aggregation of point cloud contextual information significantly enhances the representational capacity of the sparse object detection proxies. In addition, by designing sparse queries as object detection proxies, we avoid the issue of missing central features, as mentioned in SAFDNet [36]. The aforementioned results demonstrate the effectiveness of SparseDet in detecting both small and large objects.

3) Performance on the KITTI Test Set: As shown in Table III, we compare SparseDet with the SOTA methods on the KITTI *test* set. We note that our SparseDet shows outstanding performance at three difficulty levels of 3-D and BEV detection (90.79%, 81.17%, and 78.11% in 3-D APs and 94.48%, 89.68%, and 87.53% in BEV APs). It is worth noting that VoxelNeXt [35] has not provided the results of the KITTI dataset. For a fair comparison, we reproduce VoxelNeXt [35] as the baseline, following the configuration exactly as described in the original article [35]. From Table III, we understand that Our SparseDet surpasses VoxelNeXt [46] on most metrics. Especially, on the challenging hard-level tasks, we improve 1.12%, 2.72%, and 2.98% in car, pedestrian, and cyclist categories, respectively. Compared to the latest dense approach, PVT-SSD [62], our method achieves superior performance, reaching 76.81% and 63.46% on the difficult levels for the car and two-wheeler categories, respectively. Overall, our SparseDet performs well on the KITTI [42] *test* set, especially on the hard level that mostly consists of distant and small objects, strongly demonstrating the effectiveness of our method.

4) Performance on the KITTI val Dataset: We further provide the results of the KITTI *val* set to better present the detection performance of our SparseDet, as shown in Table IV.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

PERFORMANCE COMPARISON WITH THE SOTA METHODS ON THE KITTI *val* DATASET FOR CAR CLASS. THE RESULTS ARE REPORTED BY THE MAP WITH A 0.7 IOU THRESHOLD AND 40 RECALL POINTS. "SPARSE" REFERS TO METHODS THAT USE SPARSE BACKBONE WITH SPARSE DETECTION HEAD. "DENSE" REFERS TO METHODS THAT USE DENSE DETECTION HEAD

	Method	AP_{3D} (%)			AP_{BEV} (%)		
		Easy	Mod.	Hard	Easy	Mod.	Hard
Dense	$H^2$3D R-CNN [66]	89.63	85.20	79.08	-	-	-
	SECOND [25]	87.43	76.48	69.10	-	-	-
	MedTr-TSD [71]	89.27	84.24	78.85	-	-	-
	Voxel RCNN [46]	92.38	85.29	82.86	95.52	91.25	88.99
	PV-RCNN [48]	92.57	84.83	82.69	95.76	91.11	88.93
	PC-RGNN [72]	90.94	81.43	80.45	-	-	-
	PDV [73]	92.56	85.29	83.05	-	-	-
	SIENet [67]	91.96	84.45	82.64	-	-	-
	BSAODet [65]	92.27	85.06	82.75	-	-	-
Sparse	PointRCNN [28]	88.88	78.63	77.38	-	-	-
	CT3D [74]	92.85	**85.82**	83.46	**96.14**	91.88	89.63
	PG-RCNN [75]	92.73	85.26	82.83	-	-	-
	FARP-Net [76]	89.91	83.99	79.21	-	-	-
	Part-A^2 [49]	89.47	79.47	78.54	90.42	88.61	87.31
	VoxelNeXt* [35]	92.51	84.35	82.71	95.24	90.28	90.45
	SparseDet	**93.81**	84.78	**84.33**	95.30	**92.55**	**91.02**
	improvement	*+1.30*	*+0.43*	*+1.62*	*+0.06*	*+2.17*	*+0.57*

* denotes re-implement result.

TABLE V

RUNTIME COMPARISON ON NUSCENES [32] DATASET. MEM. DENOTES THE TRAINING GPU MEMORY MEASURED FOLLOWING [25]. FPS IS THE INFERENCE SPEED MEASURED ON A SINGLE NVIDIA 3090 GPU WITH A BATCH SIZE OF 1

Method	mAP	NDS	FPS	#Params	Mem.(G)
FSD [33]	62.5	68.7	10.1	11.7M	4.97
FSDV2 [34]	66.2	71.7	10.3	10.6M	4.80
VoxelNeXt [35]	64.5	70.0	15.1	7.62M	3.42
SparseDet	66.7	71.9	13.5	7.98M	3.71

There are significant improvements compared to the baseline VoxelNeXt [35] on moderate and hard levels. Meanwhile, compared to dense detection approaches, our method also demonstrates competitive performance. For sparse detectors [35] that treat a single-center feature as a physical detection proxy, insufficient aggregation contextual information weakens the ability to present objects at central points, thus limiting the performance of the detection methods. The key factor that makes SparseDet effective is its ability to comprehensively aggregate contextual information at critical locations, thereby enhancing the object proxies' capacity to present objects.

5) Comparison With Other Sparse Detectors: To demonstrate the effectiveness of our SparseDet, we conduct a comparative evaluation on the nuScenes [32] dataset by comparing SparseDet against other sparse detection frameworks [33], [34], [35] as shown in Table V. In the table, we also compare the inference speed, the size of parameters, and memory usage of SparseDet with prior sparse detection frameworks. SparseDet achieves the highest mAP, meanwhile maintaining an excellent inference speed. Compared to the baseline VoxelNeXt, SparseDet achieves a 2.2% increase in mAP performance but only has an increase of 0.36 M parameters. In addition, SparseDet does not need any auxiliary tasks. This is the main reason that SparseDet outperforms previous FSD [33] and FSDV2 [34] methods in inference speed, the size of parameters, and training memory consumption.

TABLE VI

EFFECT OF EACH COMPONENT IN OUR SPARSEDET. RESULTS ARE REPORTED ON THE KITTI *val* SET FOR CAR CATEGORY WITH VOXELNEXT. FPS IS THE INFERENCE SPEED MEASURED ON A SINGLE NVIDIA 3090 GPU WITH A BATCH SIZE OF 1

LMFA	GFA	Hard		#Params	FPS
		$AP_{3D}(\%)$	$AP_{BEV}(\%)$		
		78.44	87.10	7.45 M	20.1
✓		81.67*+3.23*	89.24*+2.14*	7.55 M	18.6
✓	✓	84.33*+2.66*	91.02*+1.78*	7.79 M	17.9

The color red indicates improvement.

TABLE VII

EFFECT OF EACH COMPONENT IN OUR SPARSEDET. RESULTS ARE REPORTED ON NUSCENES *val* SET (TRAINED ON (1/4) SUBSET) WITH VOXELNEXT. FPS IS THE INFERENCE SPEED MEASURED ON A SINGLE NVIDIA 3090 GPU WITH A BATCH SIZE OF 1

LMFA	GFA	mAP	NDS	#Params	FPS
		51.8	60.7	7.62M	15.1
✓		52.9*+1.1*	61.4*+0.7*	7.71M	14.0
✓	✓	55.3*+2.4*	62.7*+1.3*	7.98M	13.5

The color red indicates improvement.

D. Ablation Study

1) Effect of LMFA and GFA Modules: This section discusses the results of ablation experiments conducted on the baseline detector VoxelNeXt [35] to evaluate the performance of each component in SparseDet. The results are reported in Tables VI and VII for KITTI and nuScenes (1/4) subset, respectively. Table VI shows the initial AP scores for both AP_{3-D} and AP_{BEV} on KITTI, which are 78.44% and 87.10%, respectively. As shown in Table VI, LMFA and GFA modules lead to a significant improvement in performance on the hard-level KITTI tasks, with an increase of 4.27% and 3.35% for AP_{3-D} and AP_{BEV}, respectively. All the improvements do not significantly increase the model's parameter or weaken the inference speed.

As shown in Table VII, when using the LMFA module, SparseDet achieves an excellent performance improvement, which indicates that effectively aggregating contextual information can better enhance the representation ability of sparse features, thereby improving the performance of the sparse 3-D object detector. Compared to the LMFA module, which focuses on learning local details of point clouds, the GFA module dynamically learns information from global sparse features. This promotes collaboration between scene and instance features, resulting in richer and more accurate object representations. When LMFA and GFA are integrated, this enhancement effect is further amplified, leading to improvements of 2.4% in mAP and 1.3% in NDS. In summary, our ablation experiments show that SparseDet effectively enhances the performance of the baseline on challenging datasets. The research results underscore the importance

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

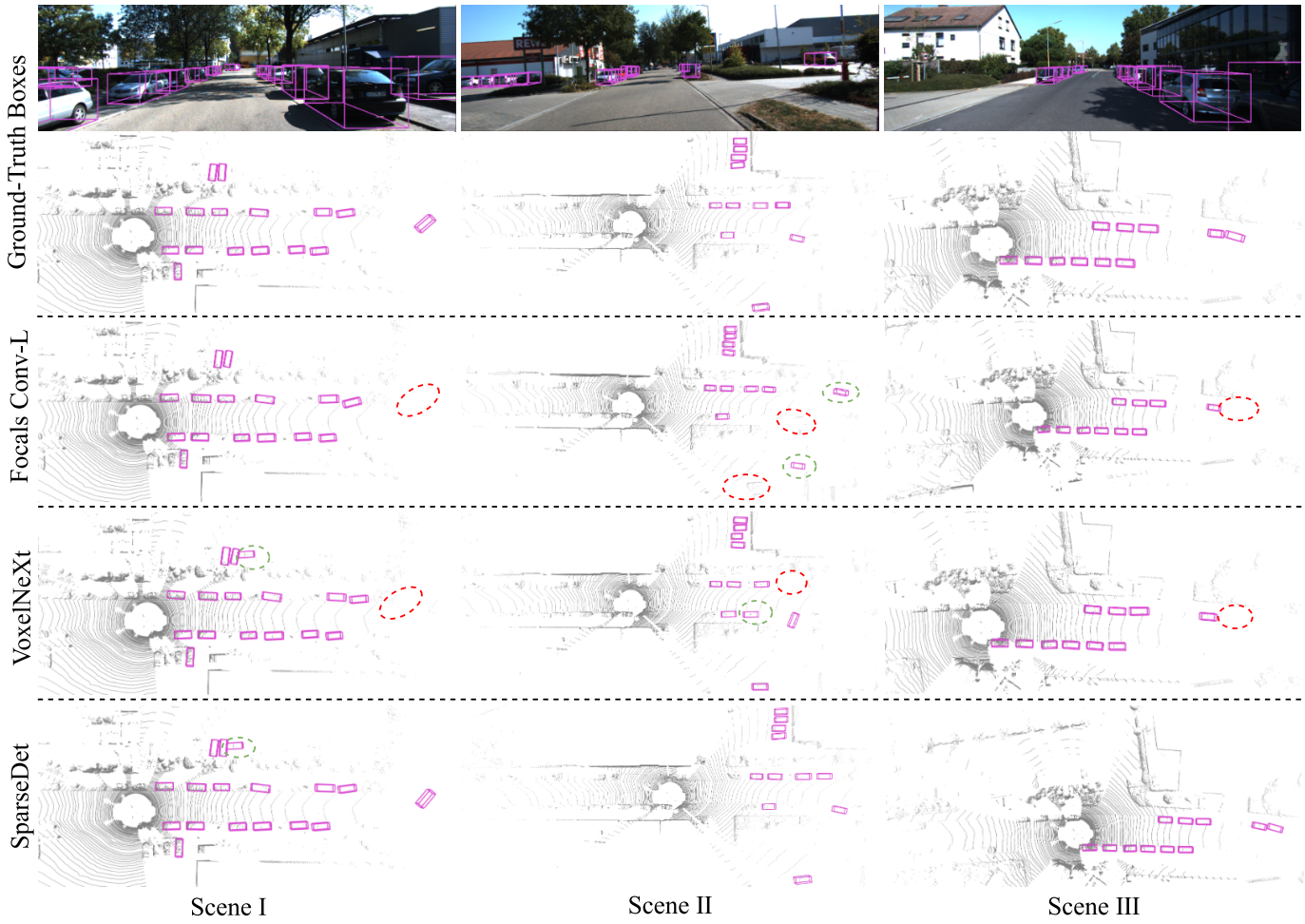


Fig. 5. Visualization comparison between VoxelNeXt, Focals Conv-L, and our SparseDet in long-range detection, where false positives and false negatives are highlighted in green and red, respectively.

of contextual information aggregation in sparse detection frameworks and provide valuable insights for designing effective aggregation strategies.

2) Effect of the Number of M: The selection of neighboring voxel features, to enhance the feature representation at key locations, is a critical component of the LMFA module. In this section, we will discuss the choice of the number of neighboring voxels M and its corresponding effectiveness. Therefore, we configure different values for the hyperparameter M (the number of neighboring voxels) including 4, 8, 16, and 32. As shown in Table VIII, the variation in the value of M does not exhibit a significant impact on the model's performance. It is worth noting that when M is set to 8, our SparseDet model achieves the highest mAP while setting M to 16 results in the best NDS performance. Considering the overall model performance, inference time, training memory, and model parameters, we ultimately set M to 8 as the default value.

3) Effect of the Number of N_{key}: As shown in Table IX, we conduct an ablation study on the number of key voxels N_{key} within the LMFA module on the nuScenes *val* dataset. We configure the value of the hyperparameter N_{key} among 500, 1000, 1500, and 2000. In summary, as the value of N_{key} increases, the performance of the SparseDet correspondingly achieves varying degrees of improvement. Another observation from the table is that the model's performance does not exhibit a strong sensitivity to the changes of N_{key}. Although

TABLE VIII
EFFECT OF THE NUMBER OF M ON NUSCENES *val* SET (TRAINED ON (1/4) SUBSET) WITH SPARSEDET

M	mAP	NDS	FPS	#params	Mem.(G)
4	54.7	62.5	13.8	7.92M	3.60
8	**55.3**	62.7	13.5	7.98M	3.71
16	55.2	**62.9**	12.9	8.11M	3.88
32	54.3	62.1	12.1	8.37M	4.01

TABLE IX
EFFECT OF THE NUMBER OF N_{key} ON NUSCENES *val* SET (TRAINED ON (1/4) SUBSET) WITH SPARSEDET

N_{key}	mAP	NDS	FPS	#params	Mem.(G)
500	55.3	62.7	13.5	7.98M	3.71
1000	55.6	**63.2**	12.9	7.98M	3.89
1500	55.6	62.6	12.0	7.98M	3.93
2000	**55.8**	63.0	11.6	7.98M	3.98

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

TABLE X

EFFECT OF THE NUMBER OF $N_{K,V}$ ON NUSCENES *val* SET (TRAINED ON (1/4) SUBSET) WITH SPARSEDET

$N_{K,V}$	mAP	NDS	FPS	#params	Mem.(G)
6000	54.1	61.7	14.2	7.98M	3.51
8000	54.9	62.3	13.8	7.98M	3.62
10000	55.3	62.7	13.5	7.98M	3.71
12000	**55.4**	**62.9**	13.0	7.98M	3.83

TABLE XI

PERFORMANCE OF DIFFERENT DISTANCES. THE RESULTS ARE EVALUATED WITH AP CALCULATED BY 40 RECALL POSITIONS AND 0.7 IOU THRESHOLD FOR THE CAR CATEGORY IN THE HARD LEVEL ON THE KITTI *val* SET

Method		$AP_{3D}(\%)$			$AP_{BEV}(\%)$		
		0-20m	20-40m	40m-inf	0-20m	20-40m	40m-inf
Dense	Voxel RCNN	93.14	73.42	25.97	92.42	86.12	51.00
	Focals Conv-L	94.25	77.27	36.59	93.00	89.22	52.34
Sparse	VoxelNeXt*	94.70	77.81	35.49	95.25	87.27	54.23
	SparseDet	95.72+1.02	79.95+2.14	44.77+9.28	96.66+1.41	89.26+1.99	63.63+9.40

* denotes re-implement result.
The color red indicates improvement to our SparseDet.

TABLE XII

PERFORMANCE OF DIFFERENT DISTANCES. THE RESULTS ARE EVALUATED WITH MAP AND NDS ON THE NUSCENES *val* SET

Method		mAP(%)			NDS(%)		
		0-20m	20-40m	40m-inf	0-20m	20-40m	40m-inf
Dense	Focals Conv-L*	75.6	48.7	22.3	75.9	48.7	43.6
	Transfusion-L*	76.2	51.5	24.1	76.3	61.2	44.0
Sparse	VoxelNeXt	75.9	49.5	22.3	76.3	60.2	42.2
	SparseDet	77.6+1.7	55.3+5.8	26.4+4.1	77.3+1.0	63.8+3.6	45.6+3.6

The color red indicates improvement to our SparseDet.

simply increasing the value of N_{key} can enhance the model's performance, it comes at the cost of reduced inference speed. After weighing the model's accuracy and inference latency, we ultimately select 500 as the default value for N_{key}.

4) Effect of the Number of $N_{K,V}$: As shown in Table X, we conduct an ablation study on the hyperparameter $N_{K,V}$ in the GFA module on the nuScenes *val* set. We configure the value of the hyperparameter $N_{K,V}$ among 6000, 8000, 10 000, and 12 000. It is worth noting that when the value of $N_{K,V}$ is set to 12 000, SparseDet achieves the highest mAP and NDS scores, but it exhibits the lowest inference speed. After weighing the model's accuracy and inference latency, we ultimately set $N_{K,V}$ to 10 000 as the default value.

5) Performance of Model in Different Distances: Compared to dense detectors, a key advantage of sparse detectors is their ability to extend the models' long-range detection capabilities without a significant increase in inference latency. Consequently, the stable detection of distant targets is a critical metric for evaluating the performance of sparse detectors. To better understand the superior performance of our SparseDet at long distances, we provide performance metrics for different distance ranges in Tables XI and XII. On the KITTI and nuScenes datasets, we compared our method not only against the baseline VoxelNeXt but also against a series of high-performance dense detectors, including Voxel RCNN, Focals Conv, and Transfusion-L. On the KITTI dataset, our method significantly outperformed the dense detectors Voxel RCNN and Focals Conv-L in long-range detection (40 m-inf), with improvements of 8.18% and 18.80% in AP_{3-D}, respectively. On the nuScenes dataset, our method also demonstrated superior detection accuracy over dense detectors Focals Conv and Transfusion-L across various distances. Specifically, in long-range detection (40 m-inf), SparseDet achieved improvements of 1.6% in NDS and 2.3% in mAP compared to the previous state-of-the-art dense detector Transfusion-L. Specifically, compared to the VoxelNeXt [35], our metrics show a more significant improvement, especially in the distance ranges of 20–40 m and 40 m-inf. For example, in 3-D detection at 40 m-inf of KITTI, our SparseDet improves AP_{3-D} by 9.28%. In BEV detection at 40 m-inf, our SparseDet improved AP_{BEV} by 9.40%. And on the nuScenes dataset, in the detection at 40 m inf, our SparseDet improves by 4.1% and 3.6% on mAP and NDS, respectively. These results clearly reflect the advantages of our SparseDet model in longer-range detection.

E. Visualization

In Fig. 5, compared to VoxelNeXt and Focals Conv-L, we illustrate the superiority of our SparseDet for long-range/remote object detection, using the detection range of 0–70.4 m for the car category in KITTI as a case. According to the figure, our SparseDet has a false-positive result, but there is no missed instance. Whereas, VoxelNeXt [35] and Focals Conv-L [51] suffer from the missing remote objects. This can be attributed to the fact that Our SparseDet fully utilizes the multiscale contextual semantic information in point clouds, which is crucial for remote objects in sparse point clouds since these objects more obviously suffer from the lack of information. Overall, our method exhibits a significant improvement in the precision of remote object detection.

V. CONCLUSION

In this work, we propose SparseDet, a simple and effective framework for fully sparse 3-D object detection. Specifically, based on VoxelNeXt, we have designed an efficient sparse detection framework that more reasonably utilizes instance-level and scene-level point cloud contextual information. This has significantly enhanced the expression capability of object proxies, thereby substantially improving the detection performance of a sparse detector. Comprehensive experimental results demonstrate that SparseDet significantly improves the performance compared with the baseline on the KITTI and nuScenes datasets. We hope our work can provide new insights into sparse detectors for autonomous driving.

Currently, the research works on sparse 3-D detectors are not as sufficient as the other directions such as multimodal 3-D detection. This makes the comparison methods of 3-D sparse frameworks limited.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] L. Wang et al., "Multi-modal 3D object detection in autonomous driving: A survey and taxonomy," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 7, pp. 3781–3798, Jul. 2023.

[2] Z. Song et al., "Robustness-aware 3D object detection in autonomous driving: A review and outlook," 2024, *arXiv:2401.06542*.

[3] Z. Song, H. Wei, L. Bai, L. Yang, and C. Jia, "GraphAlign: Enhancing accurate feature alignment by graph matching for multi-modal 3D object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 3358–3369.

[4] Z. Song, C. Jia, L. Yang, H. Wei, and L. Liu, "GraphAlign++: An accurate feature alignment by graph matching for multi-modal 3D object detection," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 4, pp. 2619–2632, Apr. 2024.

[5] S. Xu, F. Li, Z. Song, J. Fang, S. Wang, and Z.-X. Yang, "Multi-sem fusion: Multimodal semantic fusion for 3-D object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5703114.

[6] J. Bi, H. Wei, G. Zhang, K. Yang, and Z. Song, "DyFusion: Cross-attention 3D object detection with dynamic fusion," *IEEE Latin Amer. Trans.*, vol. 22, no. 2, pp. 106–112, Feb. 2024.

[7] G. Zhang, J. Xie, L. Liu, Z. Wang, K. Yang, and Z. Song, "URFormer: Unified representation LiDAR-camera 3D object detection with transformer," in *Proc. Chin. Conf. Pattern Recognit. Comput. Vis. (PRCV)*. Cham, Switzerland: Springer, 2023, pp. 401–413.

[8] X. Zhang et al., "RI-fusion: 3D object detection using enhanced point features with range-image fusion for autonomous driving," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.

[9] Z. Song et al., "RoboFusion: Towards robust multi-modal 3D object detection via SAM," 2024, *arXiv:2401.03907*.

[10] L. Wang et al., "InterFusion: Interaction-based 4D radar and LiDAR fusion for 3D object detection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 12247–12253.

[11] Z. Song et al., "GraphBEV: Towards robust BEV feature alignment for multi-modal 3D object detection," 2024, *arXiv:2403.11848*.

[12] Z. Song et al., "VoxelNextFusion: A simple, unified and effective voxel fusion framework for multi-modal 3D object detection," 2024, *arXiv:2401.02702*.

[13] B. Xu et al., "RPFA-Net: A 4D RaDAR pillar feature attention network for 3D object detection," in *Proc. IEEE Int. Intell. Transp. Syst. Conf. (ITSC)*, Sep. 2021, pp. 3061–3066.

[14] L. Wang et al., "Multi-modal and multi-scale fusion 3D object detection of 4D radar and LiDAR for autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 5628–5641, May 2023.

[15] P. Wang, L. Shi, B. Chen, Z. Hu, J. Qiao, and Q. Dong, "Pursuing 3-D scene structures with optical satellite images from affine reconstruction to Euclidean reconstruction," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5632214.

[16] S. Xu, D. Zhou, J. Fang, J. Yin, Z. Bin, and L. Zhang, "Fusionpainting: Multimodal fusion with adaptive attention for 3D object detection," in *Proc. IEEE Int. Intell. Transp. Syst. Conf. (ITSC)*, Sep. 2021, pp. 3047–3054.

[17] Y. Jiao, Z. Jie, S. Chen, J. Chen, L. Ma, and Y.-G. Jiang, "MSMDFusion: Fusing LiDAR and camera at multiple scales with multi-depth seeds for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 21643–21652.

[18] Z. Yang, Y. Sun, S. Liu, X. Shen, and J. Jia, "STD: Sparse-to-dense 3D object detector for point cloud," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1951–1960.

[19] Y. Li et al., "BEVDepth: Acquisition of reliable depth for multi-view 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 2, pp. 1477–1485.

[20] L. Yang et al., "BEVHeight: A robust framework for vision-based roadside 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 21611–21620.

[21] L. Yang et al., "BEVHeight++: Toward robust visual centric 3D object detection," 2023, *arXiv:2309.16179*.

[22] J. Huang and G. Huang, "BEVDet4D: Exploit temporal cues in multi-camera 3D object detection," 2022, *arXiv:2203.17054*.

[23] L. Yang et al., "Mix-teaching: A simple, unified and effective semi-supervised learning framework for monocular 3D object detection," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 11, pp. 6832–6844, Nov. 2023.

[24] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast encoders for object detection from point clouds," 2018, *arXiv:1812.05784*.

[25] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, Oct. 2018. [Online]. Available: http://dblp.uni-trier.de/db/journals/sensors/sensors18.html#YanML18

[26] Y. Zhou and O. Tuzel, "VoxelNet: End-to-end learning for point cloud based 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4490–4499. [Online]. Available: http://dblp.uni-trier.de/db/conf/cvpr/cvpr2018.html#ZhouT18

[27] W. Shi and R. Rajkumar, "Point-GNN: Graph neural network for 3D object detection in a point cloud," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1711–1719.

[28] S. Shi, X. Wang, and H. Li, "PointRCNN: 3D object proposal generation and detection from point cloud," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 770–779. [Online]. Available: http://dblp.uni-trier.de/db/conf/cvpr/cvpr2019.html#ShiWL19

[29] Z. Song, H. Wei, C. Jia, Y. Xia, X. Li, and C. Zhang, "VP-Net: Voxels as points for 3D object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–12, 2023, Art. no. 5701912.

[30] L. Wang et al., "SAT-GCN: Self-attention graph convolutional network-based 3D object detection for autonomous driving," *Knowl.-Based Syst.*, vol. 259, Jan. 2023, Art. no. 110080.

[31] L. Wang et al., "Fuzzy-NMS: Improving 3D object detection with fuzzy classification in NMS," *IEEE Trans. Intell. Vehicles*, pp. 1–15, 2024.

[32] H. Caesar et al., "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11621–11631.

[33] L. Fan, F. Wang, N. Wang, and Z.-X. Zhang, "Fully sparse 3D object detection," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 351–363.

[34] L. Fan, F. Wang, N. Wang, and Z. Zhang, "FSD V2: Improving fully sparse 3D object detection with virtual voxels," 2023, *arXiv:2308.03755*.

[35] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia, "VoxelNeXt: Fully sparse VoxelNet for 3D object detection and tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 21674–21683.

[36] G. Zhang, J. Chen, G. Gao, J. Li, S. Liu, and X. Hu, "SAFDNet: A simple and effective network for fully sparse 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 14477–14486.

[37] P. Sun et al., "SWFormer: Sparse window transformer for 3D object detection in point clouds," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Oct. 2022, pp. 426–442.

[38] T. Yin, X. Zhou, and P. Krahenbuhl, "Center-based 3D object detection and tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 11784–11793.

[39] G. Zhang, J. Chen, G. Gao, J. Li, and X. Hu, "HEDNet: A hierarchical encoder–decoder network for 3D object detection in point clouds," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024, pp. 53076–53089.

[40] H. Wu, J. Deng, C. Wen, X. Li, C. Wang, and J. Li, "CasA: A cascade attention network for 3-D object detection from LiDAR point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5704511.

[41] T. Wang, Z. H. U. Xinge, J. Pang, and D. Lin, "Probabilistic and geometric depth: Detecting objects in perspective," in *Proc. Conf. Robot Learn.*, 2022, pp. 1475–1485.

[42] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3354–3361. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/6248074

[43] P. Sun et al., "Scalability in perception for autonomous driving: Waymo open dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2020, pp. 2446–2454.

[44] X. Zhang et al., "Dual radar: A multi-modal dataset with dual 4D radar for autononous driving," 2023, *arXiv:2310.07602*.

[45] H. Kuang, B. Wang, J. An, M. Zhang, and Z. Zhang, "Voxel-FPN: Multi-scale voxel feature aggregation for 3D object detection from LiDAR point clouds," *Sensors*, vol. 20, no. 3, p. 704, Jan. 2020.

[46] J. Deng et al., "Voxel R-CNN: Towards high performance voxel-based 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 1201–1209.

[47] Q. Xia et al., "3-D HANet: A flexible 3-D heatmap auxiliary network for object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5701113.

[48] S. Shi et al., "PV-RCNN: Point-voxel feature set abstraction for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 10529–10538.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.

[49] S. Shi, Z. Wang, J. Shi, X. Wang, and H. Li, "From points to parts: 3D object detection from point cloud with part-aware and part-aggregation network," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 8, pp. 2647–2664, Aug. 2020.

[50] Y. Hu et al., "AFDetV2: Rethinking the necessity of the second stage for object detection from point clouds," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 1, 2022, pp. 969–979.

[51] Y. Chen, Y. Li, X. Zhang, J. Sun, and J. Jia, "Focal sparse convolutional networks for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 5428–5437.

[52] S. Vora, A. H. Lang, B. Helou, and O. Beijbom, "PointPainting: Sequential fusion for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 4604–4612.

[53] Y. Chen, S. Liu, X. Shen, and J. Jia, "Fast point R-CNN," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9775–9784.

[54] X. Bai et al., "TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 1090–1099.

[55] Y. Wang, V. C. Guizilini, T. Zhang, Y. Wang, H. Zhao, and J. Solomon, "DETR3D: 3D object detection from multi-view images via 3D-to-2D queries," in *Proc. Conf. Robot Learn.*, vol. 164, 2022, pp. 180–191.

[56] P. Gao, M. Zheng, X. Wang, J. Dai, and H. Li, "Fast convergence of DETR with spatially modulated co-attention," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 3601–3610, doi: 10.1109/ICCV48922.2021.00360.

[57] Q. Chen, L. Sun, Z. Wang, K. Jia, and A. Yuille, "Object as hotspots: An anchor-free 3D object detection approach via firing of hotspots," 2019, *arXiv:1912.12791*.

[58] J. Wang, S. Lan, M. Gao, and L. S. Davis, "InfoFocus: 3D object detection for autonomous driving with dynamic information modeling," in *Proc. Comput. Vis. 16th Eur. Conf.*, Glasgow, U.K., Aug. 2020, pp. 405–420.

[59] S. Deng, Z. Liang, L. Sun, and K. Jia, "VISTA: Boosting 3D object detection via dual cross-VIew SpaTial Attention," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 8448–8457.

[60] Y. Li, Y. Chen, X. Qi, Z. Li, J. Sun, and J. Jia, "Unifying voxel-based representation with transformer for 3D object detection," 2022, *arXiv:2206.00630*.

[61] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia, "LargeKernel3D: Scaling up kernels in 3D sparse CNNs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13488–13498.

[62] H. Yang et al., "PVT-SSD: Single-stage 3D object detector with point-voxel transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13476–13487.

[63] Z. Yang, Y. Sun, S. Liu, and J. Jia, "3DSSD: Point-based 3D single stage object detector," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11040–11048.

[64] C. Chen, Z. Chen, J. Zhang, and D. Tao, "SASA: Semantics-augmented set abstraction for point-based 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 1, pp. 221–229, doi: 10.1609/aaai.v36i1.19897.

[65] W. Xiao, Y. Peng, C. Liu, J. Gao, Y. Wu, and X. Li, "Balanced sample assignment and objective for single-model multi-class 3D object detection," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 9, pp. 5036–5048, Sep. 2023.

[66] J. Deng, W. Zhou, Y. Zhang, and H. Li, "From multi-view to hollow-3D: Hallucinated hollow-3D R-CNN for 3D object detection," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 12, pp. 4722–4734, Dec. 2021.

[67] Z. Li, Y. Yao, Z. Quan, W. Yang, and J. Xie, "SIENet: Spatial information enhancement network for 3D object detection from point cloud," 2021, *arXiv:2103.15396*.

[68] Z. Liu, X. Zhao, T. Huang, R. Hu, Y. Zhou, and X. Bai, "TANET: Robust 3D object detection from point clouds with triple attention," in *Proc. Conf. Artif. Intell. (AAAI)*, Feb. 2020, pp. 11677–11684.

[69] S. Shi et al., "PV-RCNN++: Point-Voxel feature set abstraction with local vector representation for 3D object detection," *Int. J. Comput. Vis.*, vol. 131, no. 2, pp. 531–551, Feb. 2023.

[70] C. He, R. Li, S. Li, and L. Zhang, "Voxel set transformer: A set-to-set approach to 3D object detection from point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 8417–8427.

[71] X. Tian, M. Yang, Q. Yu, J. Yong, and D. Xu, "MedoidsFormer: A strong 3D object detection backbone by exploiting interaction with adjacent medoid tokens," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 10, pp. 5844–5854, Oct. 2023.

[72] Y. Zhang, D. Huang, and Y. Wang, "PC-RGNN: Point cloud completion and graph neural network for 3D object detection," 2021, *arXiv:2012.10412*.

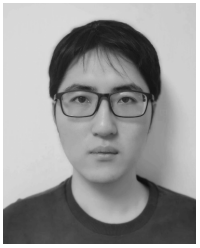
[73] J. S. K. Hu, T. Kuai, and S. L. Waslander, "Point density-aware voxels for LiDAR 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 8469–8478.

[74] H. Shenga et al., "Improving 3D object detection with channel-wise transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 2743–2752.

[75] I. Koo, I. Lee, S.-H. Kim, H.-S. Kim, W.-J. Jeon, and C. Kim, "PG-RCNN: Semantic surface point generation for 3D object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 18142–18151.

[76] T. Xie et al., "FARP-Net: Local–global feature aggregation and relation-aware proposals for 3D object detection," *IEEE Trans. Multimedia*, vol. 26, pp. 1027–1040, 2023.

[77] O. Team, "OpenPCDet: An open-source toolbox for 3D object detection from point clouds," Tech. Rep., 2020.

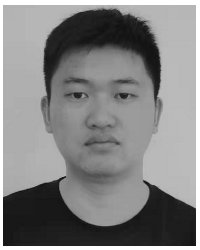


Lin Liu was born in Jinzhou, Liaoning Province, China, in 2001. He received the bachelor's degree from China University of Geosciences, Beijing, China, in 2023. He is currently pursuing the master's degree with Beijing Jiaotong University, Beijing.

His research interests include computer vision.



Ziying Song was born in Xingtai, Hebei Province, China, in 1997. He received the B.S. degree from Hebei Normal University of Science and Technology, Hebei, China, in 2019, and the master's degree from Hebei University of Science and Technology, Hebei, in 2022. He is currently pursuing the Ph.D. degree in computer science and technology with Beijing Jiaotong University, Beijing, China, with a research focus on computer vision.



Qiming Xia received the M.S. degree from Jimei University, Xiamen, China, in 2022. He is currently pursuing the Ph.D. degree with the School of Informatics, Xiamen University, Xiamen.

His research interests include computer vision, machine learning, and 3-D object detection.



Feiyang Jia was born in Yinchuan, Ningxia Province, China, in 1998. He received the B.S. degree from Beijing Jiaotong University, Beijing, China, in 2020, and the master's degree from Beijing Technology and Business University, Beijing, in 2023. He is currently pursuing the Ph.D. degree in computer science and technology with Beijing Jiaotong University, with a research focus on computer vision.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.



Caiyan Jia was born in 1976. She received the Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2004.

She was a Post-Doctoral Researcher with Shanghai Key Laboratory of Intelligent Information Processing, Fudan University, Shanghai, China, from 2004 to 2007. She is currently a Professor with the School of Computer Science and Technology, Beijing Jiaotong University, Beijing. Her research interests include deep learning in computer vision, graph neural networks, and social computing.



Lei Yang (Graduate Student Member, IEEE) received the B.E. degree from Taiyuan University of Technology, Taiyuan, China, in 2015, and the M.S. degree from the Robotics Institute, Beihang University, Beijing, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Vehicle and Mobility, Tsinghua University, Beijing.

He joined the Autonomous Driving Research and Development Department of JD.COM as an Algorithm Researcher, Beijing, from 2018 to 2020. His current research interests include computer vision, 3-D scene understanding, and autonomous driving.



Yan Gong received the M.S. degree in computer science and technology from Northeastern University, Shenyang, China, in 2023.

He was a Research Assistant with the School of Vehicle and Mobility, Tsinghua University, Beijing, China. He is now a Researcher at the Autonomous Driving Division of X Research Department, JD Logistics, Beijing. He has authored and co-authored many academic articles. His research interests include pattern recognition, multimodal fusion, and deep learning.



Hongyu Pan received the B.E. degree from Beijing Institute of Technology (BIT), Beijing, China, in 2016, and the M.S. degree in computer science from the Institute of Computing Technology (ICT), University of Chinese Academy of Sciences (UCAS), Beijing, in 2019.

He is currently at the Horizon Robotics, Beijing. His research interests include computer vision, pattern recognition, and image processing. He specifically focuses on 3-D detection/segmentation/motion and depth estimation.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 08:42:14 UTC from IEEE Xplore. Restrictions apply.